scalert_event — სრული დოკუმენტაცია

English version · გამართვის ინსტრუქცია

`scalert_event` ავტომატურად აგზავნის შეტყობინებას SeisComP-ის მიწისძვრაზე. `scalert` იძახებს ჩვენს სკრიპტს და გადასცემს `event_id`-ს. სკრიპტი SeisComP-ის ბაზიდან იღებს მიწისძვრის პარამეტრებს და წესების მიხედვით წყვეტს, ვის ეგზავნება შეტყობინება. შემდეგ აგზავნის მეილს და SMS-ს, მიწისძვრას ამატებს iesdata საიტზე და, საჭიროებისას, აქვეყნებს WordPress-ზე.

ეს დოკუმენტი აღწერს, **როგორ მუშაობს** სისტემა. ინსტალაცია, `.env`-ის შევსება და პრობლემების გადაჭრა აღწერილია გამართვის ინსტრუქციაში.

შინაარსი

1. მოკლედ
2. როგორ იძახებს scalert სკრიპტს
3. შესვლის წერტილი: `scalert_event.py`
4. მონაცემები SeisComP-ის ბაზიდან: `seiscomp_db.py`
5. გადაწყვეტილება: `automate_events/`
6. ერთი მიწისძვრის განმეორებითი გადათვლები
7. მეილი და SMS: `send_notification/`
8. iesdata საიტი და WordPress: `send_eq2api/`
9. ლოგი
10. კონფიგურაცია: `.env`
11. მიმღებები
12. ტესტები
13. ფაილების სტრუქტურა

1. მოკლედ

```
SeisComP (scautoloc / scevent / scmag ...)
        │  ახალი ან განახლებული მიწისძვრა
        ▼
scalert ──► scalert_event.py "<message>" <flag> <event_id> <arrivals> <magnitude>
                │
                ▼
          [1] არგუმენტები, ღოგი RECEIVED
                │
                ▼
          [2] seiscomp_db.load_event(event_id)       SeisComP-ის MySQL ბაზა
                │   EventData: დრო, lat, lon, სიღრმე, რეჟიმი, მაგნიტუდები
                ▼
          [3] automate_events.process_event(...)     გადაწყვეტიღება
                │      ├── rules.py                  ვის ეგზავნება და რატომ
                │      ├── location.py               ღოკაციის ტექსტი (ქართ. და ინგღ.)
                │      ├── acceleration.py           რამდენად იგრძნობა
                │      └── borders_db.py + geo.py    ies_borders ბაზა, გეომეტრია
                │   EventAssessment: audience, send_api, publish_iliauni, location, reason
                ▼  მხოღოდ თუ audience არის all ან staff
          [4] არხები, ერთმანეთისგან დამოუკიდებლად
                ├── MAIL  send_notification/ies_mail_sender   email_recipients.json
                ├── SMS   send_notification/ies_sms_sender    sms_recipients.json
                └── API   send_eq2api/event_sender            iesdata საიტი → WordPress
                │
                ▼
          [5] ღოგი DONE: თითო არხის შედეგი
```

პრინციპი: გადაწყვეტილება და გაგზავნა ცალ-ცალკეა. `automate_events` მხოლოდ წყვეტს და არაფერს აგზავნის. `send_notification` და `send_eq2api` მხოლოდ აგზავნიან და არაფერს წყვეტენ. ამიტომ ახალი წესი მხოლოდ `rules.py`-ში ემატება, ახალი არხი (მაგ. Telegram) კი ცალკე მოდულად. ერთი მეორეზე გავლენას არ ახდენს.

არხები ერთმანეთს არ აჩერებენ. თუ ერთი ჩავარდა, დანარჩენები მაინც იგზავნება, ხოლო ჩავარდნა ლოგში და გამოსვლის კოდში ჩანს.

2. როგორ იძახებს scalert სკრიპტს

`scalert` მიწისძვრის შექმნისას და ყოველი განახლებისას უშვებს `scripts.event`-ში მითითებულ სკრიპტს შემდეგი არგუმენტებით:

არგუმენტი	მნიშვნელობა
`$1`	ტექსტური შეტყობინება (scalert-ის ფორმატით)

არგუმენტი	მნიშვნელობა
`$2`	`1` — ახალი მიწისძვრა, `0` — განახლება
`$3`	მიწისძვრის publicID, მაგ. `ies2026rudf` — **ეს არის `event_id`**
`$4`	arrival-ების (ფაზების) რაოდენობა
`$5`	მაგნიტუდა (შეიძლება არ იყოს)

სკრიპტი მხოლოდ `$3`-ს ეყრდნობა. `$1`, `$2`, `$4` და `$5` ლოგში იწერება, მაგრამ გადაწყვეტილებისთვის ყველა პარამეტრი ბაზიდან იკითხება, რომ ერთ წყაროს ეყრდნობოდეს.

ხელით გაშვება (scalert-ის გარეშე):

```
./scalert_event.py ies2026rudf
./scalert_event.py --dry-run ies2026rudf     # წყვეტს და წერს ლოგს, არაფერს აგზავნის
```

3. შესვლის წერტილი: `scalert_event.py`

მხოლოდ ნაბიჯებს აერთებს, ბიზნეს-ლოგიკა აქ არ არის:

1. კითხულობს არგუმენტებს და წერს `RECEIVED`;
2. `process()`: ბაზიდან წაკითხვა → გადაწყვეტილება → არხები;
3. თითო არხს ცალკე იძახებს. ერთის შეცდომა დანარჩენებს არ აჩერებს;
4. წერს `DONE`-ს და აბრუნებს გამოსვლის კოდს.

კოდი	მნიშვნელობა
`0`	დამუშავდა: გაიგზავნა, ან გადაწყდა, რომ არ იგზავნება
`1`	შეცდომა: მიწისძვრა ბაზაში ვერ მოიძებნა, ან რომელიმე არხი ჩავარდა შეცდომით
`2`	არასწორი არგუმენტები

არხის შეცდომად ითვლება API-ის შეცდომა (`EventApiError`), მიმღებების ფაილის შეცდომა (`RecipientsError`) და ნებისმიერი მოულოდნელი შეცდომა. თუ SMTP ან SMS-ის ყველა პროვაიდერი ვერ აგზავნის, ეს ლოგში იწერება (`MAIL FAILED`, `SMS FAILED`), მაგრამ `DONE`-ში ჩანს როგორც `not_sent` და გამოსვლის კოდს არ ცვლის.

4. მონაცემები SeisComP-ის ბაზიდან: `seiscomp_db.py`

`load_event(event_id) -> EventData`

```
Event (publicID = event_id)
  ├── preferredOriginID     ──► Origin: time, latitude, longitude, depth, evaluationMode
  └── preferredMagnitudeID  ──► Magnitude (preferred)
Origin._oid ──► Magnitude._parent_oid: ამ origin-ის ყველა მაგნიტუდა (ML, MLv, M, mb, ...)
```

- გამოიყენება `Event.preferredOriginID` და არა ყველაზე ახალი origin, რადგან preferred origin-ს SeisComP თავად თვლის საუკეთესოდ;
- SQL მოთხოვნები პარამეტრიზებულია (`cursor.execute(sql, (event_id,))`);
- **რბოლის პირობა.** scalert მიწისძვრას messaging-იდან იღებს, `scdb` კი ბაზაში ასინქრონულად წერს. ამიტომ მიწისძვრა ან მისი origin შეიძლება ბაზაში ჯერ არ იყოს. `load_event` ცდილობს `SC_DB_RETRIES`-ჯერ (ნაგულისხმევად 5), `SC_DB_RETRY_DELAY` წამის (3) პაუზით, და ლოგში წერს `WAITING`. თუ ვერ იპოვა, ჩნდება `EventNotReady`, ლოგში კი ერთი ხაზი: `FAILED event_id=...: event ... not found`.

`EventData`:

ველი	ტიპი	აღწერა
`event_id`	str	publicID
`origin_id`	str	preferred origin-ის publicID
`origin_time`	datetime	UTC
`latitude`, `longitude`	float	გრადუსი
`depth`	float	კმ; თუ SeisComP-ში არ არის, `0.0`
`evaluation_mode`	str	`automatic` / `manual`
`event_type`	str | None	`not existing`, `not locatable`, `outside of network interest` → `SKIPPED`
`magnitudes`	dict[str, float]	მაგ. `{"ML": 3.4, "MLv": 3.3, "M": 3.4}`
`preferred_magnitude_type`	str | None	preferred მაგნიტუდის ტიპი

5. გადაწყვეტილება: `automate_events/`

`process_event(origin_time, lat, lon, depth, evaluation_mode, magnitudes) -> EventAssessment`

- ამოწმებს, რომ განედი, გრძედი და სიღრმე რიცხვებია;
- დრო UTC-შია; შეტყობინებისთვის ემატება +4 საათი (საქართველოს დრო);
- მაგნიტუდის არჩევა: ჯერ `ML`, თუ არ არის — `M`, თუ არც ის — პირველი არსებული; მრგვალდება ორ ათწილადამდე;
- `ies_borders` ბაზასთან კავშირი იხსნება, გამოიძახება `rules.decide`, კავშირი იხურება.

`EventAssessment`:

ველი	აღწერა
`audience`	`all`, `staff` ან `None` (არავის)
`send`	`audience is not None` — მეილი და SMS იგზავნება
`send_api`	ემატება თუ არა iesdata საიტს; `True`, როცა რომელიმე წესი დაემთხვა
`publish_iliauni`	ქვეყნდება თუ არა WordPress-ზე; `True` მხოლოდ `all`-ისთვის
`location_ge`, `location_en`	ლოკაციის ტექსტი; მხოლოდ მაშინ, როცა იგზავნება
`ev_mode`, `magnitude_type`, `magnitude`, `local_time`	შეტყობინების ტექსტისთვის
`reason`	ქართული მიზეზი რიცხვებით: რატომ მიიღო ეს გადაწყვეტილება

5.1 წესები: `rules.py`

ჯგუფი	ვინ იღებს	`send_api`	`publish_iliauni`
`all`	თანამშრომლები და გარე მიმღებები	True	True
`staff`	მხოლოდ თანამშრომლები; ტექსტს თავში აწერია `STAFF ONLY.`	True	False
არავის	—	False	False

ეპიცენტრი საქართველოშია:

პირობა	ვის ეგზავნება
მაგნიტუდა ≥ 3.0	ყველას
თბილისშია და 2.8 ≤ მაგნიტუდა < 3.0	ყველას
თბილისშია და მაგნიტუდა < 2.8	არავის

პირობა	ვის ეგზავნება
თბილისის გარეთაა და 2.8 ≤ მაგნიტუდა < 3.0	მხოლოდ თანამშრომლებს
თბილისის გარეთაა, მაგნიტუდა < 2.8, თბილისში იგრძნობა როგორც M2.8 ან მეტი	ყველას
თბილისის გარეთაა, მაგნიტუდა < 2.8, თბილისში სუსტად იგრძნობა	არავის

ეპიცენტრი საზღვარგარეთაა:

პირობა	ვის ეგზავნება
საზღვართან იგრძნობა როგორც საქართველოში მომხდარი M3.0 ან მეტი	ყველას
ასე არ არის, მაგრამ საზღვრიდან ≤ 100 კმ-ია და მაგნიტუდა ≥ 3.0	მხოლოდ თანამშრომლებს
სხვა შემთხვევაში	არავის

ზღვრები (2.8, 3.0, 100 კმ) კონსტანტებია `rules.py`-ის თავში. `send_api` და `publish_iliauni` `Decision`-ის თვისებებია, ამიტომ მათი წესის შეცვლა ერთ ადგილას ხდება.

5.2 „იგრძნობა როგორც M2.8 / M3.0“: `acceleration.py`

ითვლება, რა აჩქარებას მისცემს მიწისძვრა კონკრეტულ ადგილს (თბილისის საზღვარს ან საქართველოს საზღვრის უახლოეს წერტილს), მანძილისა და სიღრმის გათვალისწინებით. ეს დარდება „ეტალონს“, ანუ ზღვრული მაგნიტუდის მიწისძვრას, რომელიც ზუსტად იმ ადგილას მოხდა. თუ რეალური მიწისძვრა ამ ადგილს ეტალონზე ნაკლებად არ შეარყევს, ზღვარი მიღწეულია.

```
log10(a) = -4.4983 + 1.3209·M − 0.0656·M² − 0.0041·R − 0.5229·log10(R²)
```

`R` — ჰიპოცენტრული მანძილი კმ-ში (ეპიცენტრული მანძილი, სიღრმე და 3 კმ ფსევდო-სიღრმე).

5.3 ლოკაციის ტექსტი: `location.py`

იგება მხოლოდ მაშინ, როცა რამე იგზავნება, რომ ბაზაში ზედმეტი მოთხოვნები არ გაკეთდეს. ყოველი ფრაგმენტი ერთდროულად ორ ენაზე იქმნება (`Text(ge, en)`).

შემთხვევა	ქართული	ინგლისური
საქართველოში, ≥ 3.0	`საქართველო. ქაღაქი ბორჯომი, აღმოსავლეთით:11კმ. სოფელი კიმოთესუბანი:4კმ.`	`Georgia. 11 km east of Borjomi (city). Nearest settlement: Kimotesubani (village), 4 km.`
თბილისში	`საქართველო.თბიღისი, ვაკე-საბურთაღოს რაიონი.`	`Georgia. Tbilisi, Vake-Saburtalo district.`
თბილისის გარეთ, < 3.0	`საქართველო. თბიღისიდან 61კმ. ქაღაქი დმანისი, დასავლეთით:21კმ. სოფელი მამიშლარი:5კმ.`	`Georgia. 61 km from Tbilisi. 21 km west of Dmanisi (city). Nearest settlement: mamishlari (village), 5 km.`
საზღვარგარეთ	`ტაო-კლარჯეთი, საქართველოს საზღვრიდან 85კმ. დაბა შუახევი, სამხრეთით:104კმ. სოფელი წელათი:90კმ.`	`Tao-Klarjeti, 85 km from the Georgian border. 104 km south of Suaxevi (town). Nearest settlement: Tselati (village), 90 km.`

- ქართული ტექსტი ზუსტად იმეორებს ძველი სისტემის ფორმატს, ინტერვალებით და წერტილებით;
- ქალაქამდე მანძილი = ცენტრამდე მანძილი − ქალაქის რადიუსი − 1 კმ. თუ ეპიცენტრი ქალაქშია, იწერება მხოლოდ ქალაქი;
- დასახლებამდე მანძილს 1 კმ აკლდება (ძველი ფორმატი), ამიტომ ქართულში შეიძლება ეწეროს `0კმ` ან `-1კმ`; ინგლისურში უარყოფითი მანძილი `0 km` ხდება;
- მიმართულება ნაჩვენებია ქალაქიდან ეპიცენტრისკენ;
- ქალაქების და დასახლებების სახელები ორივე ენაზე იკითხება `ies_borders` ბაზიდან (`name_ge`, `name_en`, `status_ge`, `status_en`). ქვეყნების, რუსეთის რეგიონების და თბილისის რაიონების სახელები `location.py`-შია. ბაზაში ინგლისური სახელის შეცდომა (მაგ. `mamishlari`, `Suaxevi`) ბაზაშივე სწორდება, კოდის შეცვლა არ სჭირდება.

6. ერთი მიწისძვრის განმეორებითი გადათვლები

scalert სკრიპტს იძახებს **ყოველ** განახლებაზე: ახალი origin, ახალი მაგნიტუდა, მანუალური გადახედვა. ყოველი გამოძახება დამოუკიდებლად გადის ყველა ნაბიჯს. ბაზიდან ახალი მონაცემები იკითხება, წესები თავიდან ითვლება, და თუ შედეგი კრიტერიუმებს აკმაყოფილებს, შეტყობინება იგზავნება, მიუხედავად იმისა, წინა გადათვლაზე გაიგზავნა თუ არა.

არხი	ყოველ გადათვლაზე
მეილი, SMS	ახალი შეტყობინება იმ მომენტის მონაცემებით; წინა არ „უქმდება“

არხი	ყოველ გადათვლაზე
iesdata საიტი	იგივე `seiscomp_oid`-ის ჩანაწერი **ახლდება**, ახალი არ იქმნება
WordPress	`publish_iliauni=True` → ქვეყნდება ან ახლდება; `False` და ადრე გამოქვეყნებული (მაგ. `all` → `staff`) → ჩამოიხსნება

ორი გამოძახება ერთდროულად თუ მოვიდა, ორივე ცალ-ცალკე დამუშავდება.

7. მეილი და SMS: `send_notification/`

ფაილი	რას აკეთებს
`message.py`	`build_subject`, `build_body`, `build_sms_text` — მხოლოდ ტექსტი
`recipients.py`	`recipient_emails(audience)`, `recipient_numbers(audience)` — მიმღებები (ნაწილი 11)
`mailer.py`	SMTP გაგზავნა, `MAIL_*` პარამეტრებით
`ies_mail_sender.py`	`send_event_mail(...)` — ტექსტი + მიმღებები + SMTP, ლოგით
`ies_sms_sender.py`	`send_event_sms(...)` და `SmsSender` — SMS პროვაიდერების ჯაჭვით, ლოგით

7.1 მეილი

ერთი წერილი იგზავნება ყველასთან, მაგრამ მიმღებები ერთმანეთის მისამართებს ვერ ხედავენ.

თემა:

```
Earthquake - Automatic: 2026-10-07 05:15:03. mag=3.0M
```

ტექსტი (`STAFF ONLY.` მხოლოდ თანამშრომლების შემთხვევაში):

```
STAFF ONLY.
Automatic: 2026-10-07 05:15:03. mag=3.0M. ტაო-კლარჯეთი, საქართველოს საზღვრიდან 85კმ. ...
https://www.google.com/maps/place/40.68,42.27
Shakemap: https://shorturl.at/ibvtX

https://ies.iliauni.edu.ge/?page_id=121


იღიას სახელმწიფო უნივერსიტეტი
დედამიწის შემსწავლელ მეცნიერებათა ინსტიტუტი და სეისმური მონიტორინგის ეროვნული ცენტრი
```

ლოგი	მნიშვნელობა
`MAIL SENDING`	რაოდენობა, მისამართები, თემა
`MAIL SENT`	წარმატებით გადაეცა SMTP-ს
`MAIL NOT SENT`	ჯგუფში არც ერთი აქტიური მისამართი არ არის
`MAIL FAILED`	SMTP-ის შეცდომა (traceback იმავე ლოგში)
`MAIL DRY RUN`	dry run: ტექსტი ჩანს ლოგში, არაფერი იგზავნება

7.2 SMS

ტექსტი მეილის პირველი ხაზებია, როგორც ძველ სისტემაში:

```
STAFF ONLY.
Automatic: 2026-09-16 22:20:17. mag=2.92M. საქართველო. თბილისიდან 61კმ. ქალაქი დმანისი,
დასავლეთით:21კმ. სოფელი მამიშღარი:5კმ.
```

ნომრები გადაიყვანება ფორმატში `995XXXXXXXXX` (`+995 555 12-34-56` და `555123456` ორივე მისაღებია; არასწორი ნომერი ლოგში ჩანს და გამოიტოვება).

გაგზავნა ხდება პროვაიდერების ჯაჭვით (`SMS_PROVIDERS`, ნაგულისხმევად `magti,smsoffice,gsm`). ყოველი შემდეგი პროვაიდერი იღებს **მხოლოდ იმ ნომრებს, რომლებზეც წინამ ვერ გაგზავნა**:

პროვაიდერი	როგორ	წარმატება
`magti`	Magti one-way gateway, თითო ნომერზე	პასუხი `0000-<id>`, 5 წამის შემდეგ tracking სტატუსი `1`, `4` ან `8`
`smsoffice`	smsoffice.ge API, თითო ნომერზე, 3 ცდამდე	`{"Success": true}`

პროვაიდერი	როგორ	წარმატება
`gsm`	GSM მოდემი (iesresource), ერთი მოთხოვნით ყველა დარჩენილზე	HTTP 200

- პროვაიდერი, რომლის პაროლი ან გასაღები `.env`-ში ცარიელია, გამოიტოვება;
- Magti-ს დაუდასტურებელი SMS შემდეგ პროვაიდერზე გადადის, ამიტომ იშვიათად ერთი SMS შეიძლება ორჯერ მივიდეს. განგაშისთვის ეს სჯობს გამოტოვებას;
- smsoffice-ს ტექსტი 160 სიმბოლომდე აქვს; სიგრძე ლოგში `length=`-ით ჩანს;
- გაგზავნის შემდეგ მოწმდება smsoffice-ის ბალანსი; `SMSOFFICE_BALANCE_WARNING`-ზე ნაკლებისას ლოგში იწერება `SMS BALANCE LOW`.

ლოგი	მნიშვნელობა
`SMS SENDING`	ნომრები, ტექსტი და მისი სიგრძე
`SMS magti/smsoffice/gsm ...`	თითო ნომრის შედეგი თითო პროვაიდერზე
`SMS via <provider>: sent=N failed=M`	პროვაიდერის ჯამი
`SMS SENT`	ყველა ნომერზე გაიგზავნა
`SMS NOT SENT`	ჯგუფში არც ერთი აქტიური ნომერი არ არის
`SMS FAILED`	ზოგ ნომერზე ვერც ერთმა პროვაიდერმა ვერ გაგზავნა (ნომრები ჩამოთვლილია)
`SMS BALANCE LOW`	smsoffice-ზე ცოტა SMS დარჩა
`SMS DRY RUN`	dry run: არაფერი იგზავნება

8. iesdata საიტი და WordPress: `send_eq2api/`

ფაილი	რას აკეთებს
`event_sender.py`	`send_event_api(...)` — SeisComP-ის მიწისძვრა საიტის ფორმატში, publish / unpublish, ლოგით
`send_eq2api.py`	`EventApiClient` — შემოწმება, შექმნა / განახლება, წაშლა ჩავარდნისას, publish / unpublish; ასევე read-only ბრძანება

8.1 რა ხდება

```
send_api=True ?  ──არა──► არაფერი
     │ კი
     ▼
add_event: არსებობს საიტზე იგივე seiscomp_oid?
     ├── არა → POST მიწისძვრა, POST თითო მაგნიტუდა (ჩავარდნისას მიწისძვრა იშლება)
     └── კი  → PUT მიწისძვრა, PUT/POST მაგნიტუდები
     │
     ▼
publish_iliauni=True ?  ── კი ──► POST /api/publish_events/publish/{id}
     │ არა
     ▼
ადრე გამოქვეყნებული იყო (is_published)?  ── კი ──► POST /api/publish_events/unpublish/{id}
```

- **ყველაფერი ან არაფერი.** თუ ახალ მიწისძვრაზე რომელიმე მაგნიტუდა ვერ დაემატა, შექმნილი მიწისძვრა იშლება, რომ საიტზე მაგნიტუდის გარეშე არ დარჩეს. თუ წაშლაც ჩავარდა, შეცდომაში წერია, რომელი id უნდა წაიშალოს ხელით;
- **განახლებისას** თითო მაგნიტუდა ცალკე იცდება; ჩავარდნილები ერთად ჩამოითვლება. საიტზე არსებული მაგნიტუდები, რომლებიც ახალ მონაცემებში აღარ არის, არ იშლება;
- **ორი გასაღები.** მიწისძვრის დამატება `Send_eq2api_API_KEY`-ით ხდება (`can_event_edit`), publish/unpublish კი `SeisComp_Publisher_API_KEY`-ით (`can_event_publish`);
- თუ publish ან unpublish ჩავარდა, მიწისძვრა საიტზე წინა მდგომარეობაში რჩება და შემდეგი გადათვლა ისევ სცდის.

8.2 რა იგზავნება

SeisComP-ში მაგნიტუდები `{"ML": 3.512, "MLv": 3.43, "mb": 4.64, "mB": 4.9, "M": 3.512}`, საიტზე იგზავნება:

```
{
  "seiscomp_oid": "ies2026ufhk",
  "origin_time": "2026-10-12 09:35:14",
  "latitude": 41.84595,
  "longitude": 43.55186,
  "depth": 0.0,
  "magnitudes": {"ML": 3.512, "MLV": 3.43, "MB": 4.64, "M": 3.512},
  "location_ge": "საქართველო. ქაღაქი ბორჯომი, აღმოსავლეთით:11კმ. სოფელი კიმოთესუბანი:4კმ.",
  "location_en": "Georgia. 11 km east of Borjomi (city). Nearest settlement: Kimotesubani (village), 4
km.",
  "is_automatic": false
}
```

ველი	საიდან
`seiscomp_oid`	`event_id`
`origin_time`	UTC, წამებამდე (`YYYY-MM-DD HH:MM:SS`)
`latitude` , `longitude`	5 ათწილადამდე
`depth`	კმ, 3 ათწილადამდე
`magnitudes`	საიტის კოდებით, 3 ათწილადამდე (იხ. ქვემოთ)
`location_ge` , `location_en`	ნაწილი 5.3
`is_automatic`	`evaluation_mode == "automatic"`

მაგნიტუდის კოდები. საიტი იცნობს: `ML, MB, MS, MD, MW, K, MPV, MLH, MC, MLV, M` (`SITE_MAGNITUDE_CODES`). SeisComP-ის ტიპები მათ ასოების ზომის მიუხედავად ემთხვევა (`MLv` → `MLV` , `mb` → `MB` , `Mw` → `MW`). გამონაკლისები:

- `Ms_20` → `MS` ;
- `mB` (broadband body-wave) **არ იგზავნება**, რადგან ის `mb` -სგან განსხვავებული მაგნიტუდაა;
- საიტისთვის უცნობი ტიპები (მაგ. `Mwp` , `mb_Lg`) არ იგზავნება და ლოგში ჩაიწერება.

თუ საიტზე ახალი მაგნიტუდა დაემატა, ის `send_eq2api.py` -ის `SITE_MAGNITUDE_CODES` -შიც უნდა ჩაემატოს.

8.3 API

მოთხოვნა	დანიშნულება	გასაღები
`POST /api/seismic_events/filter`	ძებნა `seiscomp_oid` -ით	edit
`POST /api/seismic_events/`	შექმნა	edit
`PUT /api/seismic_events/{id}`	განახლება	edit
`DELETE /api/seismic_events/{id}`	ჩავარდნილი შექმნის წაშლა	edit
`POST /api/seismic_events/{id}/magnitudes`	მაგნიტუდის დამატება	edit
`PUT /api/seismic_events/magnitudes/{id}`	მაგნიტუდის შეცვლა	edit
`POST /api/publish_events/publish/{id}`	WordPress-ზე გამოქვეყნება ან განახლება	publisher
`POST /api/publish_events/unpublish/{id}`	WordPress-იდან ჩამოხსნა	publisher

ყველა მოთხოვნას აქვს `X-API-Key` header და timeout (`Send_eq2api_API_TIMEOUT`, 20 წამი). API-ის სრული აღწერა: `https://iesdata.iliauni.edu.ge:3030/swagger.json`.

ხელით შემოწმება (მხოლოდ კითხულობს):

```
./send_eq2api/send_eq2api.py ies2026ufhk
```

ლოგი	მნიშვნელობა
`API MAGNITUDES SKIPPED`	საიტისთვის უცნობი მაგნიტუდის ტიპები, რომლებიც გამოტოვდა
`API SENDING`	გასაგზავნი მონაცემები და `publish_iliauni`
`API CREATED` / `API UPDATED`	საიტზე დაემატა / განახლდა, `site_id`-ით
`API PUBLISHED`	WordPress-ზე გამოქვეყნდა
`API UNPUBLISHED`	WordPress-იდან ჩამოიხსნა
`API NOT SENT`	საიტისთვის ცნობილი არც ერთი მაგნიტუდა არ არის
`API DRY RUN`	dry run: არაფერი იგზავნება
`API FAILED`	API-ის შეცდომა, ერთ ხაზად

9. ლოგი

ფაილი: `logs/event_scalert.log`. ხაზის ფორმატი: `დრო LEVEL KEYWORD event_id=... ...`

ყველა ხაზს აქვს `event_id=...`, ამიტომ ერთი მიწისძვრის მთელი ისტორია ასე ჩანს:

```
grep "event_id=ies2026rudf" logs/event_scalert.log
grep " API " logs/event_scalert.log        # მხოლოდ საიტი
grep " ERROR " logs/event_scalert.log      # მხოლოდ პრობლემები
```

საკვანძო სიტყვა	ნიშნავს
`RECEIVED`	scalert-მა გამოიძახა სკრიპტი, არგუმენტებით
`WAITING`	მიწისძვრა ბაზაში ჯერ სრულად არ ჩაწერილა, ხელახლა ცდის
`INPUT`	ბაზიდან წაკითხული მონაცემები
`DECISION`	`SEND` ან `NOT SENT`, ქართული მიზეზით

საკვანძო სიტყვა	ნიშნავს
`MAIL ...`, `SMS ...`, `API ...`	არხების დეტალები (ნაწილები 7 და 8)
`SKIPPED`	არ დამუშავდა (არარეალური მიწისძვრა ან მაგნიტუდა ჯერ არ აქვს)
`DONE`	დასრულდა: თითო არხის შედეგი (`sent` / `not_sent` / `failed`) და ხანგრძლივობა; `ERROR` , თუ რომელიმე არხი ჩავარდა
`FAILED`	ვერ დამუშავდა

ყოველი გაშვება მთავრდება ზუსტად ერთით: `SKIPPED` , `DECISION ... NOT SENT` , `DONE` ან `FAILED` .

traceback მხოლოდ მოულოდნელ შეცდომაზე იწერება. მოსალოდნელი შეცდომა ერთ ხაზად იწერება: API-ის პასუხი, მიმღებების ფაილი ან ბაზაში ვერნაპოვნი მიწისძვრა.

მაგალითი, ორი გადათვლა:

```
INFO RECEIVED event_id=ies2026rudf flag=new arrivals=12 magnitude=4.2 dry_run=False message='...'
INFO INPUT event_id=ies2026rudf origin=Origin/... origin_time=2026-09-08 04:52:15 lat=41.607
lon=43.5676 depth=15.3 mode=automatic type=earthquake magnitudes={'ML': 4.23} preferred_magnitude=ML
INFO DECISION event_id=ies2026rudf SEND to=all publish_iliauni=True send_api=True magnitude=4.23ML
local_time=2026-09-08 08:52:15 location=საქართველო. ქაღაქი ახაღქაღაქი, ... reason: ეპიცენტრი
საქართველოშია და მაგნიტუდა 4.23 ≥ 3.0
INFO MAIL SENDING event_id=ies2026rudf audience=all count=2 recipients=... subject='Earthquake -
Automatic: 2026-09-08 08:52:15. mag=4.23ML'
INFO MAIL SENT event_id=ies2026rudf audience=all count=2
INFO SMS SENDING event_id=ies2026rudf audience=all count=3 numbers=... length=119 text='...'
INFO SMS via magti: sent=3 failed=0
INFO SMS SENT event_id=ies2026rudf audience=all count=3
INFO API SENDING event_id=ies2026rudf publish_iliauni=True payload={...}
INFO API CREATED event_id=ies2026rudf site_id=12
INFO API PUBLISHED event_id=ies2026rudf site_id=12 message='Publish request completed.' wp_response='1'
INFO DONE event_id=ies2026rudf mail=sent sms=sent api=sent dry_run=False duration=7.4s
INFO RECEIVED event_id=ies2026rudf flag=update arrivals=15 magnitude=4.3 ...
...
INFO API UPDATED event_id=ies2026rudf site_id=12
INFO API PUBLISHED event_id=ies2026rudf site_id=12 ...
INFO DONE event_id=ies2026rudf mail=sent sms=sent api=sent dry_run=False duration=6.9s
```

ჩავარდნის მაგალითი:

```
ERROR API FAILED event_id=ies2026test: POST /api/seismic_events/ failed with HTTP 403: სერვისს არ აქვს
საჭირო უფლება — Missing required permission: can_event_edit
ERROR DONE event_id=ies2026test mail=sent sms=sent api=failed dry_run=False duration=0.4s
```

მეორე ლოგი, `logs/update_recipients.log`, მიმღებების განახლებისაა (ნაწილი 11).

10. კონფიგურაცია: `.env`

ყველა პაროლი, გასაღები და მისამართი მხოლოდ `.env`-შია (git-ში არ ემატება), კოდში არაფერი წერია. `config.py` კითხულობს `.env`-ს და ქმნის `Settings`-ს. პროცესის გარემოს ცვლადი `.env`-ზე მაღლა დგას, ამიტომ ერთი გაშვებისთვის მნიშვნელობის შეცვლა ასე შეიძლება: `DRY_RUN=1 ./scalert_event.py ...`.

ცვლადი	ნაგულისხმევი	დანიშნულება
`SC_DB_HOST`, `SC_DB_USER`, `SC_DB_PASSWORD`, `SC_DB_NAME`	`127.0.0.1`, `sysop`, —, `seiscomp`	SeisComP-ის ბაზა
`SC_DB_RETRIES`, `SC_DB_RETRY_DELAY`	`5`, `3`	რამდენჯერ და რა პაუზით ეძებოს ჯერ ჩაუწერელი მიწისძვრა
`BORDERS_DB_HOST`, `BORDERS_DB_USER`, `BORDERS_DB_PASSWORD`, `BORDERS_DB_NAME`	`127.0.0.1`, `ies_borders`, —, `ies_borders`	საზღვრების, ქალაქების და დასახლებების ბაზა
`MAIL_SERVER`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD`	—, `587`	SMTP
`SEND_EMAIL`, `SEND_SMS`, `SEND_EQ2API`	`1`	`0` — არხი გამორთულია
`RECIPIENTS_FILE`, `SMS_RECIPIENTS_FILE`	`recipients/email_recipients.json`, `recipients/sms_recipients.json`	მიმღებების ფაილები
`Recp_Exporter_API_URL`, `Recp_Exporter_API_KEY`	iesdata	მიმღებების API
`Recp_Exporter_API_CA`, `Recp_Exporter_API_TIMEOUT`	სისტემის CA, `20`	მხოლოდ self-signed სერტიფიკატისთვის; timeout
`Send_eq2api_API_URL`, `Send_eq2api_API_KEY`	iesdata	მიწისძვრების დამატება (`can_event_edit`)

ცვლადი	ნაგულისხმევი	დანიშნულება
`SeisComp_Publisher_API_KEY`	—	WordPress-ზე გამოქვეყნება (`can_event_publish`)
`Send_eq2api_API_CA`, `Send_eq2api_API_TIMEOUT`	სისტემის CA, `20`	მხოლოდ self-signed სერტიფიკატისთვის; timeout
`SMS_PROVIDERS`, `SMS_TIMEOUT`	`magti,smsoffice,gsm`, `10`	SMS პროვაიდერების რიგი; timeout
`MAGTI_USERNAME`, `MAGTI_PASSWORD`, `MAGTI_CLIENT_ID`, `MAGTI_SERVICE_ID`	—, —, `643`, `1`	Magti
`SMSOFFICE_KEY`, `SMSOFFICE_SENDER`, `SMSOFFICE_BALANCE_WARNING`	—, `IES`, `5000`	smsoffice.ge
`GSM_SMS_URL`, `GSM_SMS_CODE`	iesresource, —	GSM მოდემი
`LOG_FILE`, `LOG_LEVEL`	`logs/event_scalert.log`, `INFO`	ლოგი; `DEBUG`-ზე ჩანს წესებში შესული მონაცემებიც
`DRY_RUN`	`0`	`1` — არაფერი იგზავნება, მხოლოდ ლოგი

ფარდობითი მისამართები (მაგ. `recipients/...`) პროექტის საქაღალდიდან ითვლება.

11. მიმღებები

მიმღებები იმართება iesdata-ს Notify გვერდიდან. `recipients/update_recipients.py` API-დან ქმნის ორ ლოკალურ ფაილს (ორივე git-ში არ ემატება):

```
iesdata API  GET /api/recips/  (X-API-Key)
     |   {"items": [{username, is_staff, is_active,
     |               emails:  [{email, is_active}],
     |               numbers: [{phone_number, is_active}]}], "total": N}
     ▼
recipients/update_recipients.py      (cron-ით, ყოველ 10 წუთში)
     |                     |
     ▼                     ▼
email_recipients.json    sms_recipients.json
     |
     ▼
send_notification/recipients.py      (ყოველ გაგზავნაზე იკითხება)
```

```
[
  {"username": "...", "email": "a@iliauni.edu.ge", "is_staff": true,  "is_active": true},
  {"username": "...", "email": "b@example.com",    "is_staff": false, "is_active": true}
]
```

```
[
  {"username": "...", "phone_number": "+995555000001", "is_staff": true, "is_active": true}
]
```

- ჩანაწერი აქტიურია მხოლოდ მაშინ, თუ აქტიურია **მიმღებიც და მისი მისამართი ან ნომერიც**;
- `staff` ჯგუფს ეგზავნება მხოლოდ `is_staff: true`; `all`-ს — ყველა აქტიურს;
- ფაილი იკითხება ყოველ გაგზავნაზე, ამიტომ განახლება მაშინვე მოქმედებს;
- ფაილი იცვლება მხოლოდ წარმატებული პასუხის შემდეგ და ატომურად (ჯერ `.tmp`, მერე გადარქმევა);
- თუ API მიუწვდომელია, **ორივე ძველი ფაილი რჩება**. თუ რომელიმე სიაში არც ერთი აქტიური ჩანაწერი არ არის, მხოლოდ ის ფაილი რჩება ძველი (ცარიელი სიის ჩასაწერად — `--allow-empty`);
- ლოგი: `logs/update_recipients.log` — `UPDATED`, `UNCHANGED`, `FAILED`.

```
./recipients/update_recipients.py           # განახლება
./recipients/update_recipients.py --dry-run  # მხოლოდ ნახვა, ფაილები არ იცვლება
```

12. ტესტები

```
./venv/bin/python -m pytest -q
```

ტესტებს არც ბაზა სჭირდება, არც SMTP, არც ქსელი: ყველა გარე სერვისი ყალბით იცვლება.

ფაილი	რას ამოწმებს
`test_scalert_event.py`	არგუმენტები, არხების დამოუკიდებლობა, `DONE`, ლოგის ფორმატი, API-ის მონაცემები
`test_send_eq2api.py`	შექმნა, განახლება, წაშლა ჩავარდნისას, publish / unpublish, შემოწმება, მაგნიტუდის კოდები
`test_send_notification.py`, `test_sms.py`	მეილისა და SMS-ის ტექსტი, პროვაიდერების ჯაჭვი
`test_seiscomp_db.py`	SeisComP-ის ბაზიდან წაკითხვა და ხელახლა ცდა
`test_update_recipients.py`	მიმღებების ფაილების განახლება
`test_config.py`	`.env`-ის წაკითხვა

13. ფაილების სტრუქტურა

```
scalert_event/
├── scalert_event.py          შესვლის წერტილი (scalert იძახებს)
├── scalert_event_ka.md       გამართვის ინსტრუქცია
├── config.py                 .env-ის წაკითხვა → Settings
├── seiscomp_db.py            load_event(event_id) -> EventData
├── .env                      პაროლები და გასაღებები (git-ში არა); ნიმუში — .env.example
├── requirements.txt          pymysql, requests, certifi, pytest
├── automate_events/          გადაწყვეტილება
│   ├── automate_event.py     process_event, EventAssessment
│   ├── rules.py              ვის ეგზავნება, send_api, publish_iliauni
│   ├── location.py           ლოკაციის ტექსტი (ქართ. და ინგლ.)
│   ├── acceleration.py       აჩქარების ფორმულა
│   ├── borders_db.py         ies_borders ბაზა
│   └── geo.py                მანძილი, მიმართულება, პოლიგონი
├── send_notification/        მეილი და SMS
│   ├── message.py            თემა, მეილისა და SMS-ის ტექსტი
│   ├── recipients.py         მიმღებები
│   ├── mailer.py             SMTP
│   ├── ies_mail_sender.py    send_event_mail
│   └── ies_sms_sender.py     send_event_sms, SmsSender (magti → smsoffice → gsm)
├── send_eq2api/              iesdata საიტი და WordPress
│   ├── event_sender.py       send_event_api (SeisComP → საიტის ფორმატი)
│   └── send_eq2api.py        EventApiClient
├── recipients/
│   ├── update_recipients.py  API-დან მიმღებების ფაილების განახლება
│   ├── email_recipients.json (git-ში არა); ნიმუში — email_recipients.example.json
│   └── sms_recipients.json   (git-ში არა); ნიმუში — sms_recipients.example.json
├── docs/                     ეს დოკუმენტაცია: .md, .html, .pdf, build_docs.py
├── logs/                     event_scalert.log, update_recipients.log
├── tests/
└── venv/                     Python გარემო (git-ში არა)
```